

Comparación de Sintaxis: Gecode vs GNUBison

Con Fundamentos Matemáticos de CSP y AST

Pipeline CSP - Motor Lógico Tipado Multidominio

Febrero 2026

Índice

1	Fundamentos Matemáticos	4
1.1	Problemas de Satisfacción de Restricciones	4
1.1.1	Definición Formal de CSP	4
1.1.2	Asignaciones y Soluciones	4
1.2	Relación con SAT y MILP	5
1.2.1	SAT como Caso Particular de CSP	5
1.2.2	MILP y Restricciones Lineales	5
1.2.3	Jerarquía de Expresividad	6
1.3	Programación con Restricciones (CP)	6
1.3.1	Definición de CP	6
1.3.2	Operadores de Propagación	6
1.3.3	Gecode como Sistema CP	7
1.4	Ubicación en la Arquitectura del Pipeline	7
1.5	Árbol de Sintaxis Abstracta (AST)	8
1.5.1	Gramática Formal	8
1.5.2	Definición Formal de AST	8
1.5.3	Propiedades del AST	8
1.5.4	AST como Término Algebraico	9
1.5.5	Relación con el Árbol de Derivación	9
1.5.6	Semántica Estructural	9
1.5.7	Ejemplo	10
1.5.8	Cadena de Procesamiento	10
1.5.9	Conclusión sobre AST	10
2	Introducción a la Comparación de Sintaxis	11
2.1	Propósito del Documento	11
2.2	Diferencias Fundamentales	11
2.3	Convenciones de este Documento	11
3	Sintaxis Gecode	12
3.1	Estructura General	12
3.2	Sección de Variables	12
3.2.1	Estructura de Variable	12
3.2.2	Tipos de Variables	12
3.2.2.1	Tipo: boolean	12
3.2.2.2	Tipo: integer	13

3.2.2.3	Tipo: numeric	13
3.2.2.4	Tipo: set	13
3.3	Sección de Restricciones (Constraints)	14
3.3.1	Estructura de Restricción	14
3.3.2	Ejemplo de Restricción	14
3.4	Sección de Funciones	14
3.4.1	Funciones Definidas por Usuario	14
4	Sintaxis GNUBison	15
4.1	Estructura General	15
4.2	Sección de Variables	15
4.2.1	Estructura de Variable	15
4.2.2	Tipos de Variables	15
4.2.2.1	Tipo: logic	15
4.2.2.2	Tipo: integer	16
4.2.2.3	Tipo: float	16
4.2.2.4	Tipo: set	16
4.3	Sección de Expresiones	16
4.3.1	Expresiones como Strings	16
4.3.2	Operadores Soportados	17
4.4	Formato de Salida	17
4.4.1	Resultado de Evaluación	17
5	Comparación Detallada	18
5.1	Tabla Comparativa: Campos de Variables	18
5.2	Tabla Comparativa: Tipos de Datos	18
5.2.1	Transformación de Tipos	18
5.2.1.1	Boolean $\rightarrow$ Logic	18
5.2.1.2	Numeric $\rightarrow$ Float	18
5.2.1.3	Set $\rightarrow$ Set (con labels)	19
5.3	Tabla Comparativa: Estructura	19
6	Gramática Formal	20
6.1	Gramática Gecode (EBNF)	20
6.2	Gramática GNUBison (EBNF)	21
7	Transformaciones	22
7.1	Algoritmo de Transformación Gecode $\rightarrow$ GNUBison	22
7.1.1	Paso 1: Transformar Variables	22
7.1.2	Paso 2: Transformar Tipos	22
7.1.3	Paso 3: Transformar Valores	22
7.1.4	Paso 4: Extraer Expresiones	23
7.2	Ejemplo Completo de Transformación	23
7.2.1	Entrada Gecode:	23
7.2.2	Salida GNUBison:	23
8	Referencia Rápida	24
8.1	Tabla de Transformación Rápida	24

8.2	Operadores Comunes	24
9	Conclusiones	25
9.1	Resumen de Diferencias	25
9.2	Uso en el Pipeline	25
9.3	Referencias	25
10	Apéndices	26
10.1	Apéndice A: Códigos de Error	26
10.2	Apéndice B: Funciones Matemáticas	26
10.2.1	Gecode (vía MiniMath)	26
10.2.2	GNUBison	26
10.3	Apéndice C: Glosario	26

Capítulo 1

Fundamentos Matemáticos

1.1 Problemas de Satisfacción de Restricciones

1.1.1 Definición Formal de CSP

Un **Problema de Satisfacción de Restricciones** (Constraint Satisfaction Problem, CSP) se define como una terna:

$$\text{CSP} = (X, D, C)$$

donde:

- $X = \{x_1, x_2, \dots, x_n\}$ es un conjunto finito de **variables**
- $D = \{D_1, D_2, \dots, D_n\}$ es un conjunto de **dominios** finitos tales que $x_i \in D_i$
- $C = \{c_1, c_2, \dots, c_m\}$ es un conjunto de **restricciones**

Cada restricción c_k está definida sobre un subconjunto de variables:

$$\text{scope}(c_k) \subseteq X$$

y especifica un conjunto permitido de tuplas:

$$R_k \subseteq \prod_{x_i \in \text{scope}(c_k)} D_i$$

1.1.2 Asignaciones y Soluciones

Una **asignación** es una función:

$$\alpha : X \rightarrow \bigcup_i D_i \quad \text{tal que} \quad \alpha(x_i) \in D_i$$

La asignación α **satisface** una restricción c_k si:

$$(\alpha(x_{i_1}), \dots, \alpha(x_{i_p})) \in R_k$$

donde $\text{scope}(c_k) = \{x_{i_1}, \dots, x_{i_p}\}$.

El CSP consiste en encontrar una asignación α tal que:

$$\forall c_k \in C, \alpha \models c_k$$

1.2 Relación con SAT y MILP

1.2.1 SAT como Caso Particular de CSP

Un problema **SAT** (Boolean Satisfiability) se define como:

$$\text{SAT} = (B, \Phi)$$

donde:

- $B = \{b_1, \dots, b_n\}$ son variables booleanas
- Φ es una fórmula booleana en CNF (Forma Normal Conjuntiva)

SAT se embebe en CSP mediante:

$$D_i = \{0, 1\} \quad \forall i$$

Cada cláusula $(l_1 \vee l_2 \vee \dots \vee l_k)$ se transforma en una restricción:

$$R = \{\mathbf{v} \in \{0, 1\}^k \mid l_1(\mathbf{v}) \vee \dots \vee l_k(\mathbf{v}) = 1\}$$

Por tanto:

$$\boxed{\text{SAT} \subset \text{CSP}}$$

1.2.2 MILP y Restricciones Lineales

Un problema **MILP** (Mixed Integer Linear Programming) se define como:

$$\begin{aligned} \min_x \quad & c^T x \\ \text{sujeto a} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \text{ o } \mathbb{R} \end{aligned}$$

MILP puede verse como:

- Variables con dominios potencialmente infinitos (enteros o reales)
- Restricciones lineales $Ax \leq b$
- Función objetivo $c^T x$ a minimizar

Si eliminamos la función objetivo y restringimos a dominios finitos:

$$\text{MILP (factibilidad)} \rightarrow \text{CSP lineal}$$

1.2.3 Jerarquía de Expresividad

La relación entre los modelos es:

$$\boxed{\text{SAT} \subset \text{CSP} \subset \text{CP}}$$

donde:

- **SAT**: variables booleanas únicamente
- **CSP**: dominios finitos generales
- **CP** (Constraint Programming): CSP + propagación + restricciones globales

Y en otra dimensión:

$$\text{MILP} \perp \text{CSP}$$

(diferentes dominios: continuo vs discreto)

Sin embargo:

$$\text{MILP}_{\text{finito}} \subset \text{CSP}$$

1.3 Programación con Restricciones (CP)

1.3.1 Definición de CP

La **Programación con Restricciones** generaliza CSP permitiendo:

- Dominios finitos, intervalos o conjuntos
- **Restricciones globales**: `alldifferent`, `cumulative`, `circuit`, etc.
- **Propagación de dominios**: reducción incremental del espacio de búsqueda
- **Búsqueda con backtracking**: exploración sistemática

1.3.2 Operadores de Propagación

Formalmente, CP extiende CSP con operadores de propagación:

$$P_k : D \rightarrow D'$$

tales que:

1. $D' \subseteq D$ (reducción de dominios)
2. $\text{Sol}(D') = \text{Sol}(D)$ (preservación de soluciones)

1.3.3 Gecode como Sistema CP

Gecode implementa CP sobre CSP finitos. Características:

- Solver de propósito general
- Más expresivo que SAT
- Restricciones globales eficientes
- API en C++ con bindings a otros lenguajes

Por tanto:

Gecode = Solver CP general

1.4 Ubicación en la Arquitectura del Pipeline

Para ubicar los sistemas en la arquitectura:

Sistema	Tipo	Características
SAT	Solver booleano	Variables $\in \{0, 1\}$
CSP	Solver de dominios finitos	Variables $\in D_i$ finito
MILP	Optimización lineal	Variables $\in \mathbb{Z} \cup \mathbb{R}$
Gecode	Solver CP	CSP + propagación + globales

1.5 Árbol de Sintaxis Abstracta (AST)

1.5.1 Gramática Formal

Sea un **lenguaje formal** definido por una gramática:

$$G = (N, \Sigma, P, S)$$

donde:

- N es un conjunto finito de **símbolos no terminales**
- Σ es un conjunto finito de **símbolos terminales**
- P es un conjunto finito de **producciones**
- $S \in N$ es el **símbolo inicial**

Cada producción tiene la forma:

$$A \rightarrow \alpha$$

con $A \in N$ y $\alpha \in (N \cup \Sigma)^*$.

1.5.2 Definición Formal de AST

Un **Árbol de Sintaxis Abstracta** es un árbol dirigido etiquetado:

$$\text{AST} = (V, E, \lambda)$$

donde:

- V es un conjunto finito de **nodos**
- $E \subseteq V \times V$ es el conjunto de **aristas** dirigidas (padre→hijo)
- $\lambda : V \rightarrow L$ es una función de **etiquetado**

con:

$$L = \mathcal{O} \cup \mathcal{T}$$

donde:

- $\mathcal{O}$ es el conjunto de **operadores** del lenguaje
- $\mathcal{T}$ es el conjunto de **operandos** (variables, constantes)

1.5.3 Propiedades del AST

Árbol enraizado: El AST posee un único nodo raíz:

$$\exists! r \in V : \nexists v \in V \text{ tal que } (v, r) \in E$$

Alcanzabilidad: Todo nodo es alcanzable desde la raíz:

$$\forall v \in V, \exists! \text{ camino } r \rightsquigarrow v$$

1.5.4 AST como Término Algebraico

Sea una **signatura algebraica**:

$$\Sigma_L = (\mathcal{O}, \text{arity})$$

donde cada operador $o \in \mathcal{O}$ tiene aridad $\text{arity}(o) = k$.

El conjunto de AST se define inductivamente como el conjunto de **términos**:

$$T(\Sigma_L, \mathcal{V})$$

tal que:

1. Si $v \in \mathcal{V}$ entonces $v \in T$ (variables son términos)
2. Si $o \in \mathcal{O}$, $\text{arity}(o) = k$ y $t_1, \dots, t_k \in T$, entonces:

$$o(t_1, \dots, t_k) \in T$$

1.5.5 Relación con el Árbol de Derivación

Sea $\text{ParseTree}(G)$ el árbol de derivación de la gramática G .

Existe una **función de abstracción**:

$$\pi : \text{ParseTree}(G) \rightarrow \text{AST}(G)$$

que elimina nodos puramente sintácticos:

- Paréntesis
- Reglas auxiliares
- Información de precedencia

1.5.6 Semántica Estructural

Sea D un **dominio semántico**.

Una **interpretación** del lenguaje es una función:

$$\llbracket \cdot \rrbracket : T(\Sigma_L, \mathcal{V}) \rightarrow D$$

definida recursivamente por:

$$\llbracket o(t_1, \dots, t_k) \rrbracket = I(o)(\llbracket t_1 \rrbracket, \dots, \llbracket t_k \rrbracket)$$

donde $I(o) : D^k \rightarrow D$ es la interpretación del operador o .

1.5.7 Ejemplo

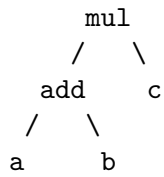
Para la expresión:

$$(a + b) \cdot c$$

el AST es el término:

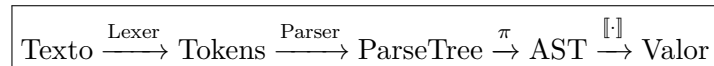
$$\text{mul}(\text{add}(a, b), c)$$

Gráficamente:



1.5.8 Cadena de Procesamiento

Todo lenguaje formal sigue la transformación:



1.5.9 Conclusión sobre AST

Un AST es la **representación algebraica canónica** de una expresión derivada de una gramática, modelada como un árbol dirigido etiquetado que preserva únicamente la **estructura operatoria esencial** del lenguaje.

Capítulo 2

Introducción a la Comparación de Sintaxis

2.1 Propósito del Documento

Este documento proporciona una comparación exhaustiva entre dos formatos de especificación de problemas lógicos:

- **Gecode**: Formato de entrada para el pipeline CSP (Constraint Satisfaction Problems)
- **GNUBison**: Formato de evaluación de expresiones con soporte de incertidumbre

Ambos sistemas operan sobre problemas de restricciones, pero con enfoques diferentes:

- **Gecode** define y **resuelve** problemas CSP
- **GNUBison** **evalúa** expresiones con valores concretos

2.2 Diferencias Fundamentales

Aspecto	Gecode	GNUBison
Propósito	Definir y resolver CSP	Evaluar expresiones
Paradigma	Solver de restricciones	Evaluador de expresiones
Incertidumbre	No soporta	Soporta múltiples valores simultáneos
Formato	JSON con AST de restricciones	JSON con variables y expresiones
Salida	Soluciones que satisfacen restricciones	Evaluación de verdad

2.3 Convenciones de este Documento

- **Código en monospace**: ejemplo
- **Palabras clave en negrita**: **boolean**, **integer**
- **Variables en MAYÚSCULAS**: TEMPERATURA, ZONA
- **Tipos en cursiva**: *logic*, *set*, *float*

Capítulo 3

Sintaxis Gecode

3.1 Estructura General

El formato Gecode usa JSON con tres secciones principales:

```
{  
  "variables": [ ... ],  
  "functions": [ ... ],  
  "constraints": [ ... ]  
}
```

3.2 Sección de Variables

3.2.1 Estructura de Variable

```
{  
  "name": "NOMBRE_VARIABLE",  
  "type": "tipo",  
  "domain": [ valores_posibles ],  
  "value": [ valores_iniciales ],  
  "id": identificador_numérico  
}
```

3.2.2 Tipos de Variables

3.2.2.1 Tipo: boolean

Descripción: Variable lógica binaria (verdadero/falso)

Representación interna: Integer 0/1 (false=0, true=1)

Ejemplo:

```
{  
  "name": "PUERTA_ABIERTA",  
  "type": "boolean",  
  "domain": [true, false],  
}
```

```

    "value": [false],
    "id": 0
}

```

3.2.2.2 Tipo: integer

Descripción: Variable de números enteros discretos

Dominio: Lista de valores enteros permitidos

Ejemplo:

```

{
  "name": "INTENTOS",
  "type": "integer",
  "domain": [0, 1, 2, 3, 4, 5],
  "value": [0],
  "id": 2
}

```

3.2.2.3 Tipo: numeric

Descripción: Variable de punto flotante (escalada $\times 1000$ internamente)

Representación: Integer escalado por `SCALE_NUM = 1000`

Ejemplo:

```

{
  "name": "TEMPERATURA_C",
  "type": "numeric",
  "domain": [-10.0, 50.0],
  "value": [20.0],
  "id": 4
}

```

Nota: Internamente se representa como 20000 (20.0×1000).

3.2.2.4 Tipo: set

Descripción: Variable de conjunto de etiquetas

Representación: Índices enteros que mapean a etiquetas (strings)

Ejemplo:

```

{
  "name": "ZONA",
  "type": "set",
  "domain": ["NORTE", "SUR", "ESTE", "OESTE"],
  "value": ["NORTE"],
  "id": 5
}

```

LabelMap: Internamente usa un mapa de índices:

```
0 → "NORTE"
1 → "SUR"
2 → "ESTE"
3 → "OESTE"
```

3.3 Sección de Restricciones (Constraints)

3.3.1 Estructura de Restricción

```
{
  "id": número,
  "expr": "expresión_string",
  "ast": { objeto_ast },
  "vars": [lista_de_ids_de_variables]
}
```

3.3.2 Ejemplo de Restricción

```
{
  "id": 0,
  "expr": "PUERTA_ABIERTA = false",
  "ast": {
    "op": "EQ",
    "left": { "type": "var", "name": "PUERTA_ABIERTA" },
    "right": { "type": "const", "value": false }
  },
  "vars": [0]
}
```

3.4 Sección de Funciones

3.4.1 Funciones Definidas por Usuario

```
{
  "name": "calcular_riesgo",
  "params": ["intentos", "nivel"],
  "body": {
    "ast": { expresión_ast },
    "expr": "intentos + nivel * 2"
  }
}
```

Capítulo 4

Sintaxis GNUBison

4.1 Estructura General

El formato GNUBison usa JSON simplificado:

```
{
  "precision": número_decimales,
  "variables": [ ... ],
  "expresiones": [ ... ]
}
```

4.2 Sección de Variables

4.2.1 Estructura de Variable

```
{
  "nombre": "NOMBRE_VARIABLE",
  "tipo": "tipo",
  "domain": [ valores ],
  "value": valor_o_valores
}
```

4.2.2 Tipos de Variables

4.2.2.1 Tipo: logic

Equivalente Gecode: boolean

Valores: true / false o 0 / 1

Ejemplo:

```
{
  "nombre": "PUERTA_ABIERTA",
  "tipo": "logic",
  "value": false
}
```


4.2.2.2 Tipo: integer

Equivalente Gecode: integer

Valores: Números enteros

Ejemplo:

```
{
  "nombre": "INTENTOS",
  "tipo": "integer",
  "value": 3
}
```

4.2.2.3 Tipo: float

Equivalente Gecode: numeric

Valores: Números de punto flotante

Ejemplo:

```
{
  "nombre": "TEMPERATURA_C",
  "tipo": "float",
  "value": 20.5
}
```

4.2.2.4 Tipo: set

Equivalente Gecode: set

Valores: Array de strings (etiquetas)

Ejemplo:

```
{
  "nombre": "ZONA",
  "tipo": "set",
  "value": ["NORTE", "SUR"]
}
```

Incertidumbre: GNUBison soporta múltiples valores simultáneos.

4.3 Sección de Expresiones

4.3.1 Expresiones como Strings

```
{
  "expresiones": [
    "PUERTA_ABIERTA = false",
    "INTENTOS + NIVEL_ALERTA <= 4",
    "TEMPERATURA_C >= 15.0 AND TEMPERATURA_C <= 25.0"
  ]
}
```

```
]
}
```

4.3.2 Operadores Soportados

Aritméticos: +, -, *, /

Comparación: =, <>, <, >, <=, >=

Lógicos: AND, OR, NOT, IMPLICA

Conjuntos: UNION, INTERSECT, DIFFERENCE, SUBSET, IN, CARDINALITY

4.4 Formato de Salida

4.4.1 Resultado de Evaluación

```
{
  "resumen": {
    "valido": true,
    "total_expresiones": 11,
    "expresiones_validas": 11,
    "expresiones_invalidas": 0,
    "errores": 0
  },
  "detalles": [
    {
      "expresion": "PUERTA_ABIERTA = false",
      "resultado": true,
      "mensaje": "Expresión válida"
    }
  ]
}
```

Capítulo 5

Comparación Detallada

5.1 Tabla Comparativa: Campos de Variables

Campo	Gecode	GNUBison	Transformación
Nombre	<code>name</code>	<code>nombre</code>	Renombrar campo
Tipo	<code>type</code>	<code>tipo</code>	Renombrar campo
Dominio	<code>domain</code>	<code>domain</code>	Sin cambios
Valor	<code>value</code>	<code>value</code>	Sin cambios
ID	<code>id</code>	-	Eliminar (no usado)

5.2 Tabla Comparativa: Tipos de Datos

Gecode	GNUBison	Conversión
<code>boolean</code>	<code>logic</code>	Cambiar nombre del tipo
<code>integer</code>	<code>integer</code>	Sin cambios
<code>numeric</code>	<code>float</code>	Dividir valor por 1000
<code>set</code>	<code>set</code>	Convertir índices a labels

5.2.1 Transformación de Tipos

5.2.1.1 Boolean $\rightarrow$ Logic

Gecode: `type="boolean", value=[0]`
GNUBison: `tipo="logic", value=false`

5.2.1.2 Numeric $\rightarrow$ Float

Gecode: `type="numeric", value=[20000]` # $\times 1000$
GNUBison: `tipo="float", value=20.0` # $\div 1000$

5.2.1.3 Set $\rightarrow$ Set (con labels)

Gecode: type="set", value=[0] # índice
 LabelMap: 0 $\rightarrow$ "NORTE"
GNUBison: tipo="set", value=["NORTE"] # label

5.3 Tabla Comparativa: Estructura

Aspecto	Gecode	GNUBison
Formato raíz	Objeto con 3 secciones	Objeto con 2-3 secciones
Variables	Array <code>variables[]</code>	Array <code>variables[]</code>
Restricciones	Array <code>constraints[]</code> con AST	Array <code>expresiones[]</code> (strings)
Funciones	Array <code>functions[]</code>	No soporta
AST	Sí, explícito	No, solo strings

Capítulo 6

Gramática Formal

6.1 Gramática Gecode (EBNF)

```
GeCodeJSON ::= "{"  
              "\"variables\":" "[" VariableList "]" ","  
              "\"functions\":" "[" FunctionList "]" ","  
              "\"constraints\":" "[" ConstraintList "]"  
              "}"
```

```
VariableList ::= Variable ("," Variable)*
```

```
Variable ::= "{"  
            "\"name\":" STRING ","  
            "\"type\":" VarType ","  
            "\"domain\":" Domain ","  
            "\"value\":" Value ","  
            "\"id\":" NUMBER  
            "}"
```

```
VarType ::= "\"boolean\"" | "\"integer\"" | "\"numeric\"" | "\"set\""
```

```
Domain ::= "[" ValueList "]"
```

```
Constraint ::= "{"  
              "\"id\":" NUMBER ","  
              "\"expr\":" STRING ","  
              "\"ast\":" AST ","  
              "\"vars\":" "[" IDList "]"  
              "}"
```

```
AST ::= "{"  
        "\"op\":" OPERATOR ","  
        "\"left\":" AST ","  
        "\"right\":" AST
```

```

    }"
| "{"
    "\"type\":" \"var\"" ", "
    "\"name\":" STRING
    }"
| "{"
    "\"type\":" \"const\"" ", "
    "\"value\":" VALUE
    }"

```

6.2 Gramática GNUBison (EBNF)

```

BisonJSON ::= "{"
    "\"precision\":" NUMBER ", "
    "\"variables\":" "[" VariableList "]" ", "
    "\"expresiones\":" "[" ExprList "]"
    }"

```

```

Variable ::= "{"
    "\"nombre\":" STRING ", "
    "\"tipo\":" VarType ", "
    "\"domain\":" Domain ", "
    "\"value\":" Value
    }"

```

```

VarType ::= "\"logic\"" | "\"integer\"" | "\"float\"" | "\"set\""

```

```

ExprList ::= STRING (", " STRING)*

```

```

Expression ::= Term (CompOp Term)*

```

```

Term ::= Factor (( "+" | "-" ) Factor)*

```

```

Factor ::= Primary (( "*" | "/" ) Primary)*

```

```

Primary ::= NUMBER
    | BOOLEAN
    | VARIABLE
    | "(" Expression ")"
    | UnaryOp Primary

```

```

CompOp ::= "=" | "<>" | "<" | ">" | "<=" | ">="

```

```

UnaryOp ::= "NOT" | "-"

```

Capítulo 7

Transformaciones

7.1 Algoritmo de Transformación Gecode $\rightarrow$ GNUBison

7.1.1 Paso 1: Transformar Variables

```
def transformar_variable(var_gecode):
    var_bison = {}
    var_bison["nombre"] = var_gecode["name"]
    var_bison["tipo"] = transformar_tipo(var_gecode["type"])
    var_bison["domain"] = var_gecode["domain"]
    var_bison["value"] = transformar_valor(
        var_gecode["value"],
        var_gecode["type"]
    )
    return var_bison
```

7.1.2 Paso 2: Transformar Tipos

```
def transformar_tipo(tipo_gecode):
    mapa = {
        "boolean": "logic",
        "integer": "integer",
        "numeric": "float",
        "set": "set"
    }
    return mapa[tipo_gecode]
```

7.1.3 Paso 3: Transformar Valores

```
def transformar_valor(valor, tipo):
    if tipo == "boolean":
        return valor[0] == 1 # 0/1  $\rightarrow$  false/true
    elif tipo == "numeric":
        return valor[0] / 1000.0 # Escala  $\div$  1000
    elif tipo == "set":
```

```

        return [label_map[idx] for idx in valor]
    else: # integer
        return valor[0]

```

7.1.4 Paso 4: Extraer Expresiones

```

def extraer_expresiones(constraints):
    return [c["expr"] for c in constraints]

```

7.2 Ejemplo Completo de Transformación

7.2.1 Entrada Gecode:

```

{
  "variables": [
    {
      "name": "TEMP",
      "type": "numeric",
      "domain": [15.0, 30.0],
      "value": [20000],
      "id": 0
    }
  ],
  "constraints": [
    {
      "id": 0,
      "expr": "TEMP >= 15.0",
      "vars": [0]
    }
  ]
}

```

7.2.2 Salida GNUBison:

```

{
  "precision": 2,
  "variables": [
    {
      "nombre": "TEMP",
      "tipo": "float",
      "domain": [15.0, 30.0],
      "value": 20.0
    }
  ],
  "expresiones": [
    "TEMP >= 15.0"
  ]
}

```


Capítulo 8

Referencia Rápida

8.1 Tabla de Transformación Rápida

Concepto	Gecode	GNUBison
Campo nombre	<code>name</code>	<code>nombre</code>
Campo tipo	<code>type</code>	<code>tipo</code>
Boolean	<code>boolean</code> (0/1)	<code>logic</code> (true/false)
Numérico	<code>numeric</code> ($\times 1000$)	<code>float</code> ($\div 1000$)
Set	Índices + LabelMap	Array de strings
Restricciones	<code>constraints[]</code> + AST	<code>expresiones[]</code> (strings)
Funciones	<code>functions[]</code>	No soporta

8.2 Operadores Comunes

Operador	Gecode	GNUBison	Tipo
Igualdad	<code>=</code>	<code>=</code>	Comparación
Desigualdad	<code><></code>	<code><></code>	Comparación
Suma	<code>+</code>	<code>+</code>	Aritmético
Resta	<code>-</code>	<code>-</code>	Aritmético
AND lógico	<code>AND</code>	<code>AND</code>	Lógico
OR lógico	<code>OR</code>	<code>OR</code>	Lógico
NOT lógico	<code>NOT</code>	<code>NOT</code>	Lógico
Pertenencia	<code>IN</code>	<code>IN</code>	Conjuntos

Capítulo 9

Conclusiones

9.1 Resumen de Diferencias

1. **Gecode** es un sistema de **modelado y resolución** de CSP
2. **GNUBison** es un **evaluador** de expresiones con valores concretos
3. Gecode usa **AST explícito**, GNUBison usa **expresiones como strings**
4. GNUBison soporta **incertidumbre** (múltiples valores simultáneos)
5. La transformación entre formatos es **directa** pero requiere:
 - Renombrado de campos
 - Conversión de tipos
 - Escalado de valores numéricos
 - Mapeo de índices a etiquetas (sets)

9.2 Uso en el Pipeline

El pipeline CSP utiliza:

1. **Gecode** para **resolver** el CSP y encontrar soluciones
2. **GNUBison** para **verificar** que las soluciones satisfacen las expresiones originales
3. **VerifyWithBison** como componente que:
 - Lee soluciones de Gecode
 - Transforma al formato GNUBison
 - Invoca el evaluador
 - Reporta verificación

Esta arquitectura proporciona **validación cruzada** de resultados.

9.3 Referencias

- Gecode: <https://www.gecode.org/>
- GNUBison: `/home/rodo/GNUBison/`
- Pipeline CSP: `/home/rodo/gecode/`
- Documentación VerifyWithBison: `docs/VerifyWithBison.txt`

Capítulo 10

Apéndices

10.1 Apéndice A: Códigos de Error

Código	Descripción	Sistema
1	Error de sintaxis JSON	Ambos
2	Tipo de variable inválido	Ambos
3	Dominio vacío	Gecode
4	Variable no declarada	GNUBison
5	Expresión inválida	GNUBison

10.2 Apéndice B: Funciones Matemáticas

10.2.1 Gecode (vía MiniMath)

- `sin(x)`, `cos(x)`, `tan(x)`
- `sqrt(x)`, `pow(x, y)`
- `log(x)`, `exp(x)`
- `abs(x)`, `min(x,y)`, `max(x,y)`

10.2.2 GNUBison

- Operadores aritméticos básicos
- Operadores lógicos
- Operadores de conjuntos
- No soporta funciones trigonométricas (aún)

10.3 Apéndice C: Glosario

- **AST**: Abstract Syntax Tree (Árbol de Sintaxis Abstracta)
- **CP**: Constraint Programming (Programación con Restricciones)
- **CSP**: Constraint Satisfaction Problem (Problema de Satisfacción de Restricciones)
- **EBNF**: Extended Backus-Naur Form (Forma de Backus-Naur Extendida)

- **MILP**: Mixed Integer Linear Programming (Programación Lineal Entera Mixta)
 - **SAT**: Boolean Satisfiability Problem (Problema de Satisfacibilidad Booleana)
-

Fin del Documento